

✓ Actividad: Git + uv sobre un proyecto real

Materia: Matematica Aplicada a ML con Programacion — USFQ **Duracion estimada:** 30-40 min

Vas a **clonar un repositorio de Machine Learning**, migrar su entorno a `uv`, ejecutar el modelo base y **mejorarlo trabajando en una rama de Git**. Practicarás el flujo completo: `git clone`, `uv`, ramas, commits y `merge`.

Repositorio: `https://github.com/mlflow/mlflow-example` — un modelo de sklearn (ElasticNet) que predice la calidad de un vino a partir de sus propiedades químicas.

Los comandos de terminal van en bloques `bash` (copialos a tu terminal / Git Bash de VS Code). Las celdas de **codigo Python** se pueden ejecutar directamente en Colab o VS Code.

Contenido

1. Clonar el repositorio
2. Explorar el proyecto
3. Migrar el entorno a `uv` (version sin mlflow)
4. Ejecutar el modelo base (baseline)
5. Crear una rama para tu mejora
6. Mejorar el modelo
7. Commit y merge
8. Ver la historia del proyecto
9. Publicar en tu GitHub
10. (Opcional) Practica: Learn Git Branching
11. Entregable
12. Referencia rapida de comandos Git

1. Clonar el repositorio

`git clone` descarga una copia completa del repositorio (con toda su historia de commits) a tu computadora.

```
git clone https://github.com/mlflow/mlflow-example.git
cd mlflow-example
```

2. Explorar el proyecto

```
ls                # archivos del proyecto
git status        # rama actual y cambios
git log --oneline # historial de commits del repo
cat conda.yaml    # dependencias que necesita
```

Archivos clave: `train.py` (entrena el modelo), `wine-quality.csv` (dataset), `conda.yaml` (dependencias).

3. Migrar el entorno a `uv`

```
uv init --bare
uv add pandas numpy scikit-learn
```

Tradicional (pip/conda)

```
conda env create -f conda.yaml
```

```
pip install -r requirements.txt
```

```
python train.py
```

Con uv

```
uv add pandas numpy scikit-learn
```

```
uv sync
```

```
uv run python train.py
```

`uv` crea `.venv`, escribe `pyproject.toml` y genera `uv.lock`.

Importante: si te da error por rutas largas en Windows

El `train.py` original del repo usa la librería **mlflow**. En **Windows** puede fallar con rutas largas (error `Can't find Python file`)

...c48cb773bb87..._for_mysql.py), porque la ruta supera los 260 caracteres). Para el objetivo de esta actividad (Git + uv + mejorar un modelo) **no necesitamos mlflow**.

Si te aparece ese error: copia el siguiente código y **reemplaza** con el todo el contenido de `train.py`. Si `uv run python train.py 0.5 0.5` te funciona sin problemas, omite este paso.

```
# Modelo de calidad de vino (ElasticNet) - version SIN mlflow
import os
import sys
import warnings

import numpy as np
import pandas as pd
from sklearn.linear_model import ElasticNet
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

def eval_metrics(actual, pred):
    rmse = np.sqrt(mean_squared_error(actual, pred))
    mae = mean_absolute_error(actual, pred)
    r2 = r2_score(actual, pred)
    return rmse, mae, r2

if __name__ == "__main__":
    warnings.filterwarnings("ignore")
    np.random.seed(40)

    wine_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                              'data', 'winoquality.csv')
    data = pd.read_csv(wine_path)

    train, test = train_test_split(data, random_state=40)
    train_x = train.drop(["quality"], axis=1)
    test_x = test.drop(["quality"], axis=1)
    train_y = train["quality"]
    test_y = test["quality"]

    alpha = float(sys.argv[1]) if len(sys.argv) > 1 else 0.5
```

```

l1_ratio = float(sys.argv[2]) if len(sys.argv) > 2 else 0.5

lr = ElasticNet(alpha=alpha, l1_ratio=l1_ratio, random_state=42)
lr.fit(train_x, train_y)
pred = lr.predict(test_x)

rmse, mae, r2 = eval_metrics(test_y, pred)
print(f"ElasticNet (alpha={alpha}, l1_ratio={l1_ratio})")
print(f"  RMSE: {rmse:.4f}")
print(f"  MAE : {mae:.4f}")
print(f"  R2  : {r2:.4f}")

```

Alternativa: en vez de quitar mlflow, mueve el proyecto a una ruta corta (ej. `C:\dev\mlflow-example`) para no superar el limite de Windows.

4. Ejecutar el modelo base (baseline)

`train.py` acepta dos argumentos: `alpha` y `l1_ratio`.

```
uv run python train.py 0.5 0.5
```

Salida esperada:

```

ElasticNet (alpha=0.5, l1_ratio=0.5)
  RMSE: 0.8222
  MAE : 0.6279
  R2  : 0.1268

```

Anota estas metricas — son tu punto de comparacion.

5. Crear una rama para tu mejora

```
git switch -c feature/mejora-modelo
```

Cualquier cambio y commit queda aislado hasta que hagas `merge`.

✓ 6. Mejorar el modelo

El objetivo es **bajar el RMSE** / subir el R2. Prueba una de estas opciones editando `train.py`.

Opcion A — Ajustar hiperparametros (sin editar codigo):

```
uv run python train.py 0.1 0.1
uv run python train.py 0.01 0.5
```

Opcion B — Escalar las features (edita `train.py`): agrega estas lineas **despues** de separar `train_x / test_x` y **antes** de entrenar el modelo:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
train_x = scaler.fit_transform(train_x)
test_x = scaler.transform(test_x)
```

Opcion C — Cambiar de modelo (edita `train.py`): agrega el import arriba y **reemplaza** las lineas donde se crea y entrena `ElasticNet`:

```
from sklearn.ensemble import RandomForestRegressor

# reemplaza: lr = ElasticNet(alpha=alpha, l1_ratio=l1_ratio, random_s
lr = RandomForestRegressor(n_estimators=200, random_state=42)
lr.fit(train_x, train_y)
pred = lr.predict(test_x)
```

Despues de cada cambio, vuelve a correr `uv run python train.py` y compara el RMSE con el baseline.

Cuando edites `train.py`, vuelve a ejecutarlo para confirmar la mejora:

```
uv run python train.py
```

Luego ejecuta `git status` para ver **que archivos modificaste** (deberia aparecer `train.py` como modificado):

```
git status
```

7. Commit y merge

Antes de commitear, revisa `git status`. Veras `train.py` modificado y archivos nuevos como `pyproject.toml`, `uv.lock` y `mlflow.db`.

Ojo: no todo debe versionarse. `mlflow.db` (y `.venv/`) son archivos generados: ignoralos con un `.gitignore` **en la raiz del proyecto** (junto a `train.py`, NO dentro de `.venv/`).

Forma 1 — desde la terminal (los comandos `echo` escriben el archivo por ti):

```
echo ".venv/" >> .gitignore
echo "mlflow.db" >> .gitignore
echo "mlruns/" >> .gitignore
echo "__pycache__/" >> .gitignore
```

Forma 2 — creando el archivo a mano: crea un archivo llamado `.gitignore` en la raiz y pega SOLO estas rutas (sin los `echo`, esos son solo para la terminal):

```
.venv/
mlflow.db
mlruns/
__pycache__/
```

Que ignora cada linea:

- `.venv/` → todo el entorno virtual
- `mlflow.db` → la base de datos local de MLflow
- `mlruns/` → los experimentos generados por MLflow

- `__pycache__/` → los archivos temporales de Python

Ahora prepara los cambios y commitea:

```
git add .
git status                # confirma que mlflow.db ya NO aparece

git commit -m "feat: mejorar el modelo + ignorar archivos generados"

git status                # tras el commit, debe decir "nothing"

# Ver los nombres de las ramas (la actual sale con *)
git branch

# Integrar en la rama principal
git switch master          # usa el nombre que veas: 'master' o '
git merge feature/mejora-modelo
```

Si `mlflow.db` ya te salía en verde (porque hiciste `git add .` antes de crear el `.gitignore`), quitálo del seguimiento con `git rm --cached mlflow.db` y vuelve a `git status`.

`pyproject.toml` y `uv.lock` si se versionan: definen tu entorno reproducible para que otros hagan `uv sync`.

8. Ver la historia del proyecto

```
git log --oneline --graph --all
```

`--graph` dibuja las ramas y el punto de `merge`.

9. (Opcional) Publicar en tu GitHub

```
# Crea un repo vacio en github.com y copia su URL, luego:
git remote set-url origin https://github.com/TU-USUARIO/mi-wine-model.
git push -u origin master
```

10. (Opcional) Practica: Learn Git Branching

- Enlace: https://learngitbranching.js.org/?locale=es_ES
- **Objetivo:** completar los primeros **8 niveles** del modulo principal.

11. Entregable

1. Captura de `git log --oneline --graph --all` con tu **commit de mejora** y el **merge**.
2. **RMSE / R2 antes vs. despues** de tu mejora.
3. (Opcional) Captura de los **8 niveles completados** en Learn Git Branching.

12. Referencia rapida de comandos Git

Comando	Que hace
<code>git clone URL</code>	Descarga un repositorio completo
<code>git status</code>	Estado actual: rama, cambios y staging
<code>git log --oneline --graph</code>	Historial visual con ramas
<code>git switch -c rama</code>	Crea una rama y se cambia a ella
<code>git add .</code>	Prepara los cambios para el commit
<code>git commit -m "msg"</code>	Crea un commit
<code>git switch master</code>	Vuelve a la rama principal
<code>git merge rama</code>	Integra <code>rama</code> en la rama actual
<code>git remote set-url origin URL</code>	Cambia el remoto
<code>git push -u origin master</code>	Sube la rama al remoto

Actividad · Matematica Aplicada a ML con Programacion (USFQ)

